

PHƯƠNG PHÁP CHUỖI TAYLOR

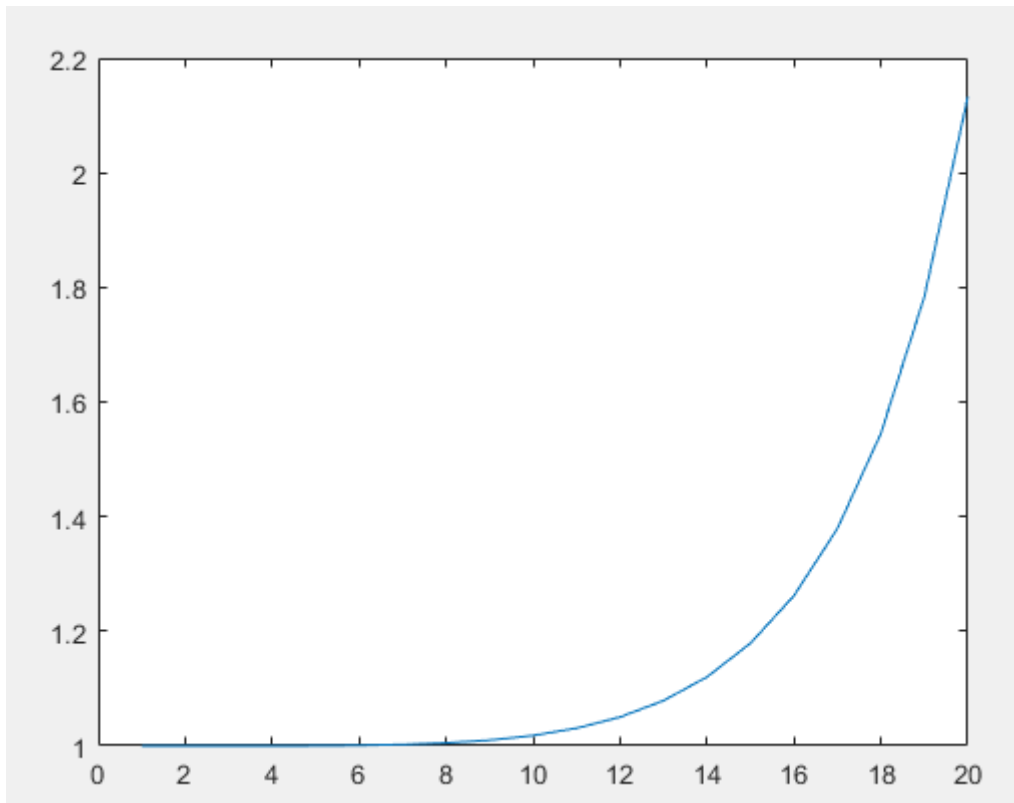
1.1. Thuật toán.

```
function [y1] = TaylorMethod(f,inter,y0,k)
t(1) = inter(1); %initial time
y1(1) = y0; y2(1) = 0; %setting the initial condition
h = 0.1*(2.^-k); % setting the step size
n = 1/h; % number of steps in terms of the step size.
Ft = @(t,y) diff(f,t);
Fy = @(t,y) diff(f,y);
for i = 1:n
    t(i+1) = t(i) + h;
    euler_y(i+1) = y(i) + h*f(t(i),y(i));
    y1(i+1) = euler_y(i+1) + ((h^2)/2)*(Ft(t(i),y1(i)) +
Fy(t(i),y1(i))*f(t(i),y1(i)));
end

clc; clear all;
f = @(t,y)(5*(t^4)*y);%      --> this is function 'f'
inter = [0 1];
y0 = 1;
k=1;
Out = TaylorMethod(f,inter,y0,k);
plot(Out);

function [y1] = TaylorMethod(f,inter,y0,k)
syms t y tp yp
Ft1=subs(diff(f,t),{t,y},{tp,yp});
Fy1=subs(diff(f,y),{t,y},{tp,yp});
clear t y;
t(1) = inter(1); %initial time
y1(1) = y0;
y2(1) = 0; %setting the initial condition
h = 0.1*(2.^-k); % setting the step size
n = 1/h; % number of steps in terms of the step size.
for i = 1:n-1
    t(i+1) = t(i) + h;
    euler_y(i+1) = y1(i) + h*f(t(i),y1(i));
    y1(i+1) = euler_y(i+1) + ((h^2)/2)*(subs(Ft1,{tp,yp},
{t(i),y1(i)}) +
    subs(Fy1,{tp,yp},{t(i),y1(i)})*f(t(i),y1(i)));
end
end
```

Kết quả:



Hình 1. Đồ thị hàm số xấp xỉ theo phương pháp Taylor.

PHƯƠNG PHÁP EULER

2.1. Thuật toán.

2.1.1. Phương pháp Euler.

```
from math import *
import numpy as np
from prettytable import PrettyTable
import matplotlib.pyplot as plt

def f(x):
    return np.round(np.sqrt(2*x + 1), 5)
def df(x, y):
    return y - 2*x/y
## Ví dụ 1
x = [0]
y = [1]
dy = [df(x[-1], y[-1])] # đạo hàm tại (xi,yi)
half_dy = [] # giá trị df_i+1/2
half_y = [] # y_i+1/2
half_x = [] # x_i+1/2
h = 0.2
it = 10
for i in range(it):
    half_x.append(x[-1] + h/2)
    half_y.append(y[-1] + h/2*dy[-1])
    half_dy.append(df(half_x[-1], half_y[-1]))
    x.append(x[-1] + h)
    y.append(y[-1] + h*half_dy[-1])
    dy.append(df(x[-1], y[-1]))

table = PrettyTable()
table.add_column("i", range(it))
table.add_column("x", np.round(x[:-1], 5))
table.add_column("y", np.round(y[:-1], 5))
table.add_column("f_i", np.round(dy[:-1], 5))
table.add_column("f_(i+1/2)", np.round(half_dy, 5))
table.add_column("y_(i+1/2)", np.round(half_y, 5))
table.add_column("y_(i+1)", np.round(y[1:], 5))
table.add_column("f(x)", f(np.array(x[:-1])))
print(table)
```

Kết quả

i	x	y	f _i	f _(i+1/2)	y _(i+1/2)	y _(i+1)	f(x)
0	0.0	1.0	1.0	0.91818	1.1	1.18364	1.0
1	0.2	1.18364	0.84569	0.7951	1.26821	1.34266	1.18322
2	0.4	1.34266	0.74682	0.71179	1.41734	1.48501	1.34164
3	0.6	1.48501	0.67694	0.65106	1.55271	1.61522	1.48324
4	0.8	1.61522	0.62465	0.60479	1.67769	1.73618	1.61245
5	1.0	1.73618	0.58423	0.56871	1.79461	1.84992	1.73205
6	1.2	1.84992	0.55257	0.54048	1.90518	1.95802	1.84391
7	1.4	1.95802	0.528	0.51889	2.01082	2.0618	1.94936
8	1.6	2.0618	0.50976	0.50352	2.11277	2.1625	2.04939
9	1.8	2.1625	0.49776	0.49459	2.21228	2.26142	2.14476

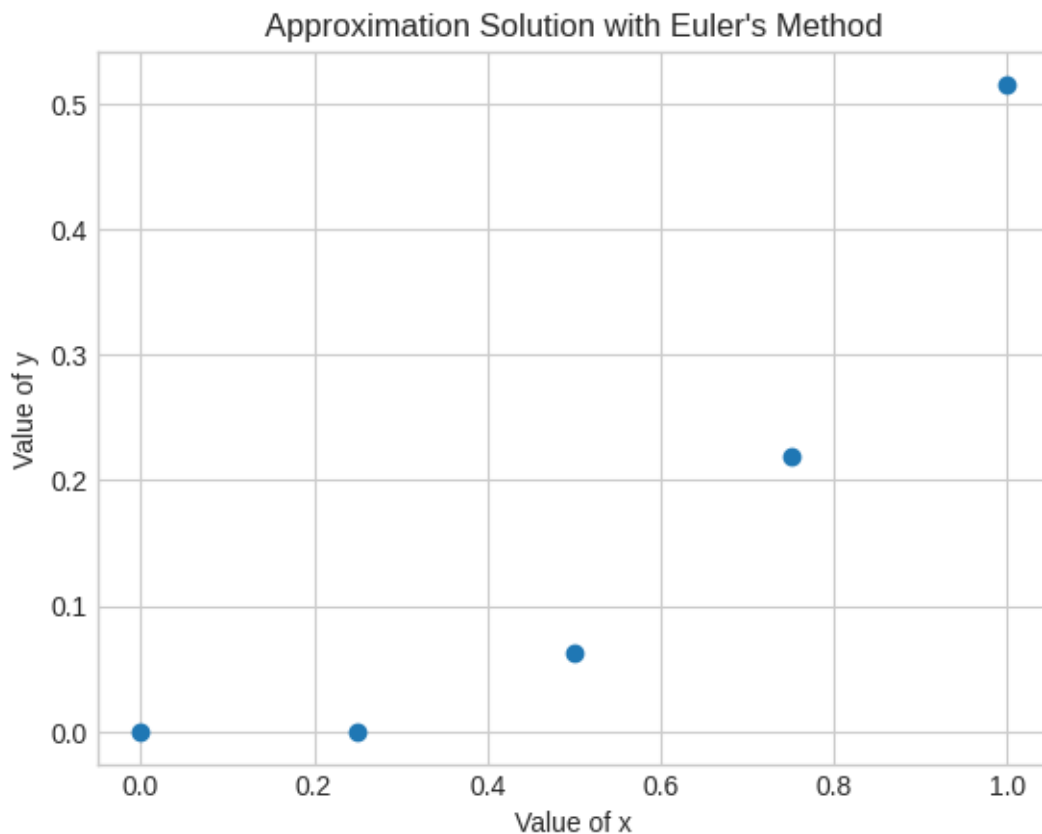
```

x0 = 0
y0 = 0
xf = 1
n = 5
deltax = (xf-x0)/(n-1)
x = np.linspace(x0,xf,n)
def f(x,y):
    return x+2*y
y = np.zeros([n])
y[0] = y0
for i in range(1,n):
    y[i] = deltax*f(x[i-1],y0) + y0
    y0 = y[i]
print("x_n\t    y_n")
for i in range(n):
    print(x[i],"\t",format(y[i],'6f'))
plt.plot(x,y,'o')
plt.xlabel("Value of x")
plt.ylabel("Value of y")
plt.title("Approximation Solution with Euler's Method")
plt.show()

```

Kết quả:

x_n	y_n
0.0	0.000000
0.25	0.000000
0.5	0.062500
0.75	0.218750
1.0	0.515625



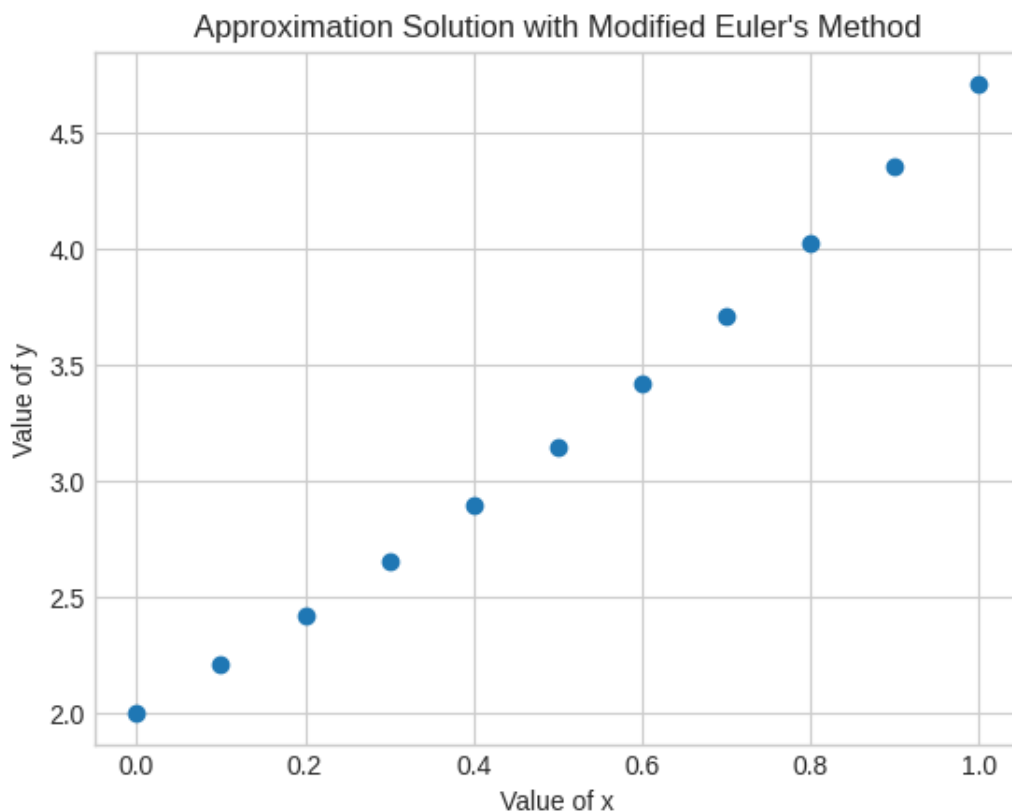
Hình 2. Biểu diễn giá trị xấp xỉ với phương pháp Euler.

2.1.2. Phương pháp Euler cải tiến.

```
x0 = 0      y0 = 2      xf = 1      n = 11      deltax = (xf-x0)/(n-1)
x = np.linspace(x0,xf,n)
def f(x,y):
    return y-x
y = np.zeros([n])      y[0] = y0      py = np.zeros([n])      py[0] = None
for i in range(1,n):
    py[i] = deltax*f(x[i-1],y[i-1]) + y[i-1]
    y[i] = deltax/2*( f(x[i],py[i]) + f(x[i-1],y[i-1]) ) + y[i-1]
print("x_n\t      py_n\t      y_n")
for i in range(n):
    print (x[i],"\t",format(py[i], '6f'),"\t",format(y[i], '6f'))
plt.plot(x,y,'o')
plt.xlabel("Value of x")
plt.ylabel("Value of y")
plt.title("Approximation Solution with Modified Euler's Method")
plt.show()
```

Kết quả:

x_n	py_n	y_n
0.0	nan	2.000000
0.1	2.200000	2.205000
0.2	2.415500	2.421025
0.3	2.643128	2.649233
0.4	2.884156	2.890902
0.5	3.139992	3.147447
0.6	3.412191	3.420429
0.7	3.702472	3.711574
0.8	4.012731	4.022789
0.9	4.345068	4.356182
1.0	4.701800	4.714081



Hình 3. Kết quả xấp xỉ với phương pháp Euler cải tiến.

PHƯƠNG PHÁP HIỆU CHỈNH ĐIỂM GIỮA

3.1. Thuật toán.

```
function [t,y] = midpoint_ivp(fun,tspan,y0,dt)
% midpoint_ivp: Solves IVP using the Midpoint method.
% [t,y] = euler_ivp(fun,t0,y0,tN,dt):
%   Solves a system of differential equations  $y' = f(t,y)$  using the
%   midpoint method (Explicit)
% input:
%   fun = fun is a function handle. For a scalar t and a vector y,
%         fun(t,y) must return a column vector corresponding to f(t,y).
%   tspan = A vector containing the the start time and end time, [t0 tN].
%   y0 = Initial value of y at t0
%   dt = Step size
% output:
%   t = Values of t
%   y = Values of y calculated at all t

    t          = tspan(1):dt:tspan(2);
    nt         = numel(t);
    y          = zeros(nt, numel(y0));
    y(1,:)     = y0;

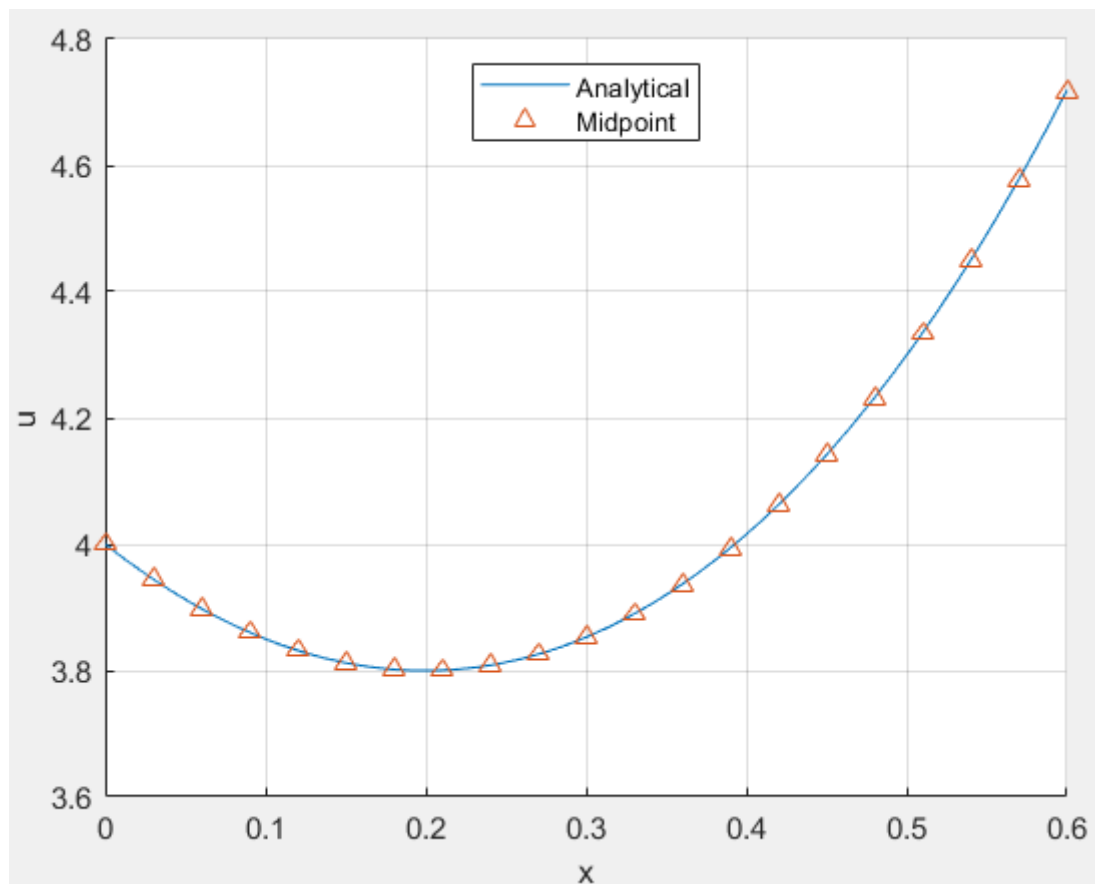
    for i = 1:nt-1
        ymid = y(i,:) + fun(t(i),y(i,:))' * dt / 2;
        y(i+1,:) = y(i,:) + fun(t(i)+dt/2,ymid)'*dt;
    end
end

-----

clear; clc; close all;
% Description: Solves an IVP using the Midpoint method
% Analytical Solution
yan = @(x) exp(-2*x).*(2*exp(x) + exp(4*x) + 1);
xspan = [0,0.6];
hold on
fplot(yan,xspan,'DisplayName','Analytical')
odefun = @(t,y) [    y(2);
                  y(3);
                  -y(3)+4*y(2)+4*y(1)    ];

y0 = [4,-2,10];
% Midpoint Method
dx = 0.03;
[t,y] = midpoint_ivp(odefun,xspan,y0,dx);
plot(t,y(:,1),'^','DisplayName','Midpoint')
xlabel('x')
ylabel('u')
grid on
legend('Location','North')
```

Kết quả:



Hình 4. Kết quả xấp xỉ bằng phương pháp điểm giữa.

PHƯƠNG PHÁP HIỆU CHỈNH HEUN

4.1. Thuật toán.

```
def f(x):
    return np.round(np.sqrt(2*x + 1), 5)
def df(x, y):
    return y - 2*x/y

x = [0]
y = [1]
dy = [df(x[-1], y[-1])]
h = 0.2
y_tmp = [] # giá trị tham số trung gian y_{i+1} = y_i + h*df(x_i, y_i)
dy_tmp = [] # giá trị tham số trung gian dy(x_{i+1}, y_{tmp[-1]})
it = 10
for i in range(it):
    y_tmp.append(y[-1] + h*dy[-1])
    x.append(x[-1] + h)
    dy_tmp.append(df(x[-1], y_tmp[-1]))
    y.append(y[-1] + h*(dy[-1] + dy_tmp[-1])/2)
    dy.append(df(x[-1], y[-1]))

table = PrettyTable()
table.add_column("i", range(it))
table.add_column("x", np.round(x[:-1], 5))
table.add_column("y", np.round(y[:-1], 5))
table.add_column("dy(xi, yi)", np.round(dy[:-1], 5))
table.add_column("y_tmp(i+1)", np.round(y_tmp, 5))
table.add_column("dy_tmp(x(i+1), y_tmp(i+1))", np.round(dy_tmp, 5))
table.add_column("y_(i+1)", np.round(y[1:], 5))
table.add_column("f(x)", f(np.array(x[:-1])))

print(table)
```

Kết quả

i	x	y	dy(xi, yi)	y_tp(i+1)	dy_t(x(i+1), y_(i+1))	y_(i+1)	f(x)
0	0.0	1.0	1.0	1.2	0.86667	1.18667	1.0
1	0.2	1.18667	0.84959	1.35658	0.76687	1.34831	1.18322
2	0.4	1.34831	0.75498	1.49931	0.69894	1.4937	1.34164
3	0.6	1.4937	0.69033	1.63177	0.65124	1.62786	1.48324
4	0.8	1.62786	0.64498	1.75686	0.61846	1.7542	1.61245
5	1.0	1.7542	0.61409	1.87702	0.5984	1.87545	1.73205
6	1.2	1.87545	0.59576	1.99461	0.59082	1.99411	1.84391
7	1.4	1.99411	0.58998	2.11211	0.59703	2.11281	1.94936
8	1.6	2.11281	0.59824	2.23246	0.61989	2.23463	2.04939
9	1.8	2.23463	0.62362	2.35935	0.66397	2.36338	2.14476

PHƯƠNG PHÁP RUNGE-KUTTA

5.1. Thuật toán.

```
def f(x):  
    return np.round(1/5*x*np.exp(3*x) - 1/25*np.exp(3*x) +  
1/25*np.exp(-2*x), 5)  
  
def dy(x, y):  
    return x*exp(3*x) - 2*y  
y = [0] # khai báo y0  
x = [0]  
h = 0.1  
it = 10 # số vòng lặp  
  
k1 = []  
k2 = []  
k3 = []  
k4 = []  
delta_y = []  
for i in range(it):  
    ki_1 = h*dy(x[-1], y[-1])  
    ki_2 = h*dy(x[-1] + h/2, y[-1] + ki_1/2)  
    ki_3 = h*dy(x[-1] + h/2, y[-1] + ki_2/2)  
    ki_4 = h*dy(x[-1] + h, y[-1] + ki_3)  
  
    delta_yi = 1/6*(ki_1 + 2*ki_2 + 2*ki_3 + ki_4)  
  
    k1 = np.append(k1, ki_1)  
    k2 = np.append(k2, ki_2)  
    k3 = np.append(k3, ki_3)  
    k4 = np.append(k4, ki_4)  
    delta_y = np.append(delta_y, delta_yi)  
  
    y = np.append(y, y[-1] + delta_yi)  
    x = np.append(x, x[-1] + h)  
  
x = np.round(x, 3)  
y = np.round(y, 5)  
k1 = np.round(k1, 5)  
k2 = np.round(k2, 5)  
k3 = np.round(k3, 5)  
k4 = np.round(k4, 5)  
delta_y = np.round(delta_y, 5)  
  
dat_dtype = {  
    'names' : ("x", "y", "k1", "k2", "k3", "k4", "delta_y", "y_i+1"),  
    'formats' : ("float", "float", "float", "float", "float",  
"float", "float", "float")}  
table = PrettyTable()  
table.add_column("x", x[:-1])  
table.add_column("y", y[:-1])  
table.add_column("k1", k1)
```

```

table.add_column("k2", k2)
table.add_column("k3", k3)
table.add_column("k4", k4)
table.add_column("delta_y", delta_y)
table.add_column("y_i+1", y[1:])

print(table)

```

Kết quả:

x	y	k1	k2	k3	k4	delta_y	y_i+1
0.0	0.0	0.0	0.00581	0.00523	0.01245	0.00575	0.00575
0.1	0.00575	0.01235	0.02114	0.02026	0.03124	0.02106	0.02682
0.2	0.02682	0.03108	0.04445	0.04312	0.0598	0.04434	0.07116
0.3	0.07116	0.05956	0.07983	0.0778	0.10301	0.07964	0.1508
0.4	0.1508	0.10265	0.13316	0.13011	0.1679	0.13285	0.28364
0.5	0.28364	0.16736	0.21292	0.20836	0.26458	0.21242	0.49606
0.6	0.49606	0.26377	0.33128	0.32453	0.40751	0.33048	0.82654
0.7	0.82654	0.40632	0.50564	0.49571	0.6174	0.5044	1.33094
0.8	1.33094	0.61567	0.76085	0.74633	0.92372	0.75896	2.0899
0.9	2.0899	0.9212	1.13224	1.11114	1.36835	1.12938	3.21928

PHƯƠNG PHÁP ADAMS BASHFORTH FOURTH ORDER

5.1. Thuật toán.

```
function adams_bashforth_method(a,b,s,f,y0)
% s is the number of subinterval for the interpolation
k=3; % steps-1
x=linspace(a,b, s+1); %network set creation
h=1/(s+1); %discretization of the network set
u=zeros(1,s+1); %network function creation
u(1)=y0; %initial condition of Cauchy problem
for i=1:k
    k_1=f(x(i),u(i)); %k_1
    k_2=f(x(i)+h./2,u(i)+((h./2).*k_1)); %k_2
    k_3=f(x(i)+h./2,u(i)+((h./2).*k_2)); %k_3
    k_4=f(x(i)+h,u(i)+(h.*k_3)); %k_4
    u(i+1)=u(i)+(h/6).*(k_1+2.*(k_2+k_3)+k_4); %Runge-Kutta
end
%Now that I know the initial four condition, as I set the step constant, I
%can evaluate the others u's by AB4 formula
for i=4:s
    u(i+1) = u(i) + h/24*(55*f(x(i),u(i))-59*f(x(i-1),
        u(i-1))+37*f(x(i-2),u(i-2))-9*f(x(i-3),u(i-3)))
end
plot(x,u,'r');
title('Adams Bashforth 4th order method')
hold on
end
-----
%% Adams-Bashworth-Moulton Predictor, corrector and other methods
%% initialization
%%
syms x y
% Range of x
xspan=[2,3];
% Step size
h=0.1
% Initial values
y0=3, x0=2
y(1)=y0;x(1)=x0;
% No. of steps
steps=(xspan(2)-xspan(1))/h;
%% Define f(x,y)
%%
f= @(x,y)x*(y-x);
%% Exact Solution
%%
[Xe,Yexact]=ode45(f,xspan,y0);
Xe=Xe(1:4:end);
Yexact=Yexact(1:4:end);
Exact_Sol=vpa([Xe Yexact],5)
%% Euler's Method
%%
for j=2:steps+1
    x(j,1)=x(j-1)+h;
    y(j,1)=y(j-1,1)+h*f(x(j-1,1),y(j-1,1));
end
euler=vpa([x y],5)
```

```

% Mean Square Error
mse = vpa(norm(Yexact-y,2),5)
%% Heun's Method
%%
for j=2:steps+1
    x(j,1)=x(j-1)+h;

    k1(j-1,1)=h*f( x(j-1), y(j-1) );

    k2(j-1,1)=h*f( x(j-1)+h, y(j-1)+k1(j-1));

    y(j,1)=y(j-1)+0.5*(k1(j-1)+k2(j-1));
end
heuns=vpa([x y],5)
% Mean Square Error
mse = vpa(norm(Yexact-y,2),5)
%% RK 4th order method
%%
for j=2:steps+1
    x(j,1)=x(j-1)+h;

    k1(j-1,1)=h*f( x(j-1), y(j-1) );

    k2(j-1,1)=h*f( x(j-1)+h/2, y(j-1)+0.5*k1(j-1) );

    k3(j-1,1)=h*f( x(j-1)+h/2, y(j-1)+0.5*k2(j-1) );

    k4(j-1,1)=h*f( x(j-1)+h, y(j-1)+k3(j-1) );

    y(j,1)=y(j-1)+(1/6)*(k1(j-1)+2*k2(j-1)+2*k3(j-1)+k4(j-1));
end
rk4=vpa([x y],5)
% Mean Square Error
mse = vpa(norm(Yexact-y,2),5)
%% Adam-Bashworth predictor
%%
for k=5:steps+1
    x(k,1)=x(k-1)+h;

    y(k,1)=y(k-1) +(h/24)*( -9*f(x(k-4),y(k-4)) +37*f(x(k-3),y(k-3))...
        -59*f(x(k-2),y(k-2)) +55*f(x(k-1),y(k-1)));
end
p=y;
Adam_Bashworth=vpa([x p],5)
% Mean Square Error
mse = vpa(norm(Yexact-y,2),5)
%% Adam-Moulton corrector
%%
for k=5:steps+1
    x(k,1)=x(k-1)+h;

    y(k,1)=y(k-1) +(h/24)*( f(x(k-3),y(k-3)) -5*f(x(k-2),y(k-2))...
        +19*f(x(k-1),y(k-1)) +9*f(x(k),p(k)));
end
Adam_Moulton=vpa([x y],5)
% Mean Square Error
mse = vpa(norm(Yexact-y,2),5)
%%
% Created by- Bhartendu Thakur, Machine Learning & Computing

```

Kết quả

```
>> adamsbashforth
```

```
h =          y0 =          x0 =  
0.1000          3          2
```

Exact_Sol =	euler =
[2.0, 3.0]	[2.0, 3.0]
[2.1, 3.2164]	[2.1, 3.2]
[2.2, 3.4726]	[2.2, 3.431]
[2.3, 3.7814]	[2.3, 3.7018]
[2.4, 4.1611]	[2.4, 4.0242]
[2.5, 4.6366]	[2.5, 4.4141]
[2.6, 5.2432]	[2.6, 4.8926]
[2.7, 6.0306]	[2.7, 5.4886]
[2.8, 7.0696]	[2.8, 6.2416]
[2.9, 8.4618]	[2.9, 7.2052]
[3.0, 10.354]	[3.0, 8.4537]

```
mse =  
2.5235
```

heuns =	rk4 =
[2.0, 3.0]	[2.0, 3.0]
[2.1, 3.2155]	[2.1, 3.2164]
[2.2, 3.4701]	[2.2, 3.4726]
[2.3, 3.7765]	[2.3, 3.7814]
[2.4, 4.1522]	[2.4, 4.1611]
[2.5, 4.6216]	[2.5, 4.6365]
[2.6, 5.2186]	[2.6, 5.2431]
[2.7, 5.9909]	[2.7, 6.0305]
[2.8, 7.0063]	[2.8, 7.0694]
[2.9, 8.3614]	[2.9, 8.4615]
[3.0, 10.195]	[3.0, 10.353]

mse =	mse =
0.20447	0.00066539

Adam_Bashworth =

[2.0, 3.0]

[2.1, 3.2164]

[2.2, 3.4726]

[2.3, 3.7814]

[2.4, 4.1605]

[2.5, 4.6348]

[2.6, 5.2395]

[2.7, 6.0238]

[2.8, 7.0576]

[2.9, 8.4413]

[3.0, 10.319]

mse =

0.042467

Adam_Moulton =

[2.0, 3.0]

[2.1, 3.2164]

[2.2, 3.4726]

[2.3, 3.7814]

[2.4, 4.1611]

[2.5, 4.6365]

[2.6, 5.2428]

[2.7, 6.0297]

[2.8, 7.0675]

[2.9, 8.4575]

[3.0, 10.345]

mse =

0.0097564